

Software Life-Cycle Processes

Waterfall, Incremental, Iterative, Spiral, V-Model, Prototyping, Agile and Scrum

SW Process Management – Week 2 | Lecturer: Professor S. Vasanthapriyan

Learning Outcomes

At the end of this lecture, students should be able to:

- ☐ **Explain the purpose of a software life cycle.**
- ☐ **Describe major software life-cycle models.**
- ☐ **Compare predictive, iterative and adaptive approaches.**
- ☐ **Explain Scrum roles, events and artefacts.**
- ☐ **Evaluate the strengths and limitations of each model.**
- ☐ **Select an appropriate life-cycle model for a given project.**

What Is a Software Life Cycle?

A software life cycle describes the stages through which a software system passes from its initial idea to retirement.

01

Feasibility and Planning

02

Requirements Analysis

03

System Design

04

Development

05

Testing

06

Deployment

07

Operation and Maintenance

08

Retirement or Replacement

What Is a Life-Cycle Model?

A software life-cycle model defines the activities, sequence, deliverables and controls that structure a development project.



Activities

The activities performed during development and their sequence



Deliverables

The outputs and artefacts produced at each stage



Stakeholders

How involvement and communication are organised



Change Control

How changes are handled and risks are managed

Key point: A life-cycle model provides structure, but it does not guarantee project success.

Why Does Model Selection Matter?

An inappropriate life-cycle model may result in:

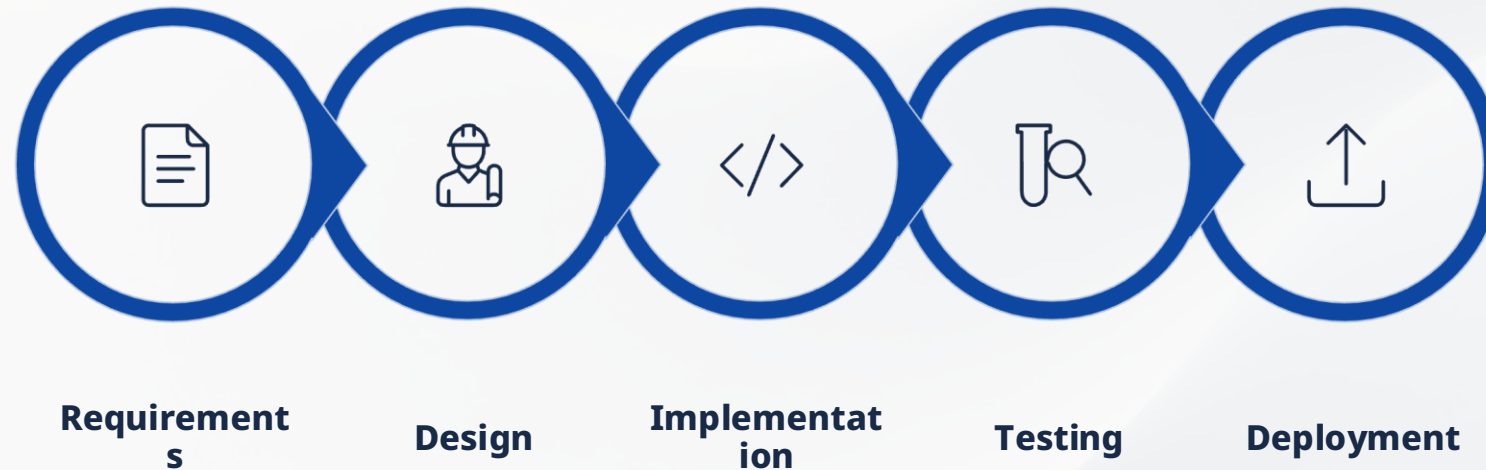
- **Requirements being misunderstood**
- **Late discovery of defects**
- **Inability to manage changes**
- **Unrealistic schedules**
- **Poor stakeholder involvement**
- **Excessive documentation**
- **Insufficient project control**
- **Increased cost and project risk**

The model should fit the project rather than forcing the project to fit the model.

WATERFALL MODEL

The Waterfall Model

The Waterfall model follows a sequential progression through defined development stages.



Each stage produces defined deliverables. A stage is usually completed before the next begins. Documentation is used to communicate between stages, and changes are formally controlled.

WATERFALL MODEL

Waterfall Activities and Deliverables

Stage	Main Activity	Example Deliverable
Requirements	Define system needs	Requirements specification
Design	Define architecture and components	Design document
Implementation	Develop the system	Source code
Testing	Verify and validate the system	Test report
Deployment	Release the system	Operational system
Maintenance	Correct and improve the system	Updated versions

Waterfall: Strengths and Limitations

Strengths

- | Simple and easy to understand
- | Clearly defined stages and milestones
- | Strong documentation
- | Easier management and progress tracking
- | Suitable for stable and well-understood requirements
- | Supports regulated and contract-driven environments

Limitations

- | Working software is delivered relatively late
- | Changes may be difficult and costly
- | Users may have limited involvement after requirements
- | Problems may be discovered during late testing
- | Unsuitable for highly uncertain projects

V-MODEL

The V-Model Connects Development and Testing

The V-Model extends the Waterfall approach by connecting each development activity with a corresponding testing activity.

Development Activity	Corresponding Test Activity
User requirements	Acceptance testing
System requirements	System testing
Architecture design	Integration testing
Component design	Unit testing
Coding	Software implementation

Key point: Testing is planned during development rather than only after coding.

Verification and Validation

Verification

"Are we building the product correctly?"

- Reviews requirements, designs and code
- Checks compliance with specifications
- May not require executing the software

Validation

"Are we building the correct product?"

- Evaluates the operational software
- Checks whether user needs are satisfied
- Involves executing and testing the system

V-MODEL

When Is the V-Model Appropriate?

Suitable When

- Requirements are stable and clearly documented
- Quality and reliability are critical
- Testing must be planned early
- Formal approval is required at each stage
- The project operates in a regulated environment

Examples

Medical software

Banking applications

Government information systems

Embedded control systems

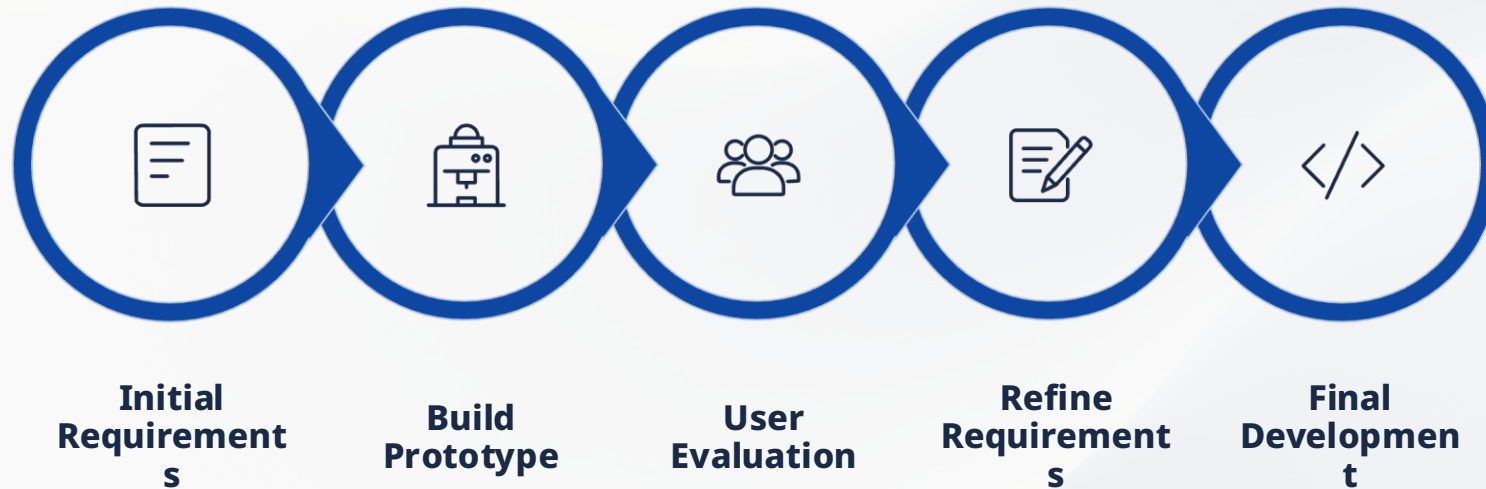
Safety-related applications

Limitation: It remains relatively inflexible when requirements change frequently.

PROTOTYPING MODEL

Prototyping Makes Requirements Visible

A prototype is an early and limited representation of a proposed software system.



Prototypes may be used to explore user-interface design, functional requirements, system feasibility, technical solutions, and user expectations.

Types of Prototyping



Throwaway Prototyping

- Created to understand requirements
- Built quickly
- Discarded after learning
- Final system is developed separately



Evolutionary Prototyping

- Begins with a basic operational system
- Continuously improved
- Eventually becomes the final product



Incremental Prototyping

- Different parts are prototyped separately
- The parts are later integrated

Prototyping: Strengths and Limitations

Strengths

- | Clarifies unclear requirements
- | Encourages early user involvement
- | Improves usability
- | Identifies misunderstandings early
- | Supports innovation and experimentation
- | Reduces the risk of developing the wrong system

Limitations

- | Users may believe the prototype is nearly complete
- | Developers may use poor temporary designs in the final system
- | Documentation may be neglected
- | Repeated changes may increase cost
- | Project scope may become difficult to control

INCREMENTAL MODEL

Incremental Development Delivers in Parts

In the Incremental model, the system is divided into a series of functional releases. Each increment delivers usable functionality.

Example: University Student-Services System

1

Increment 1

Student login and profile

2

Increment 2

Course registration

3

Increment 3

Examination results

4

Increment 4

Attendance monitoring

5

Increment 5

Online payments

INCREMENTAL MODEL

How Incremental Development Works

01

Define the overall system requirements

02

Prioritise the required features

03

Design the high-level architecture

04

Develop the first group of features

05

Test and deliver the increment

06

Collect feedback

07

Develop and integrate the next increment

Key point: The product grows through a sequence of completed releases.

Incremental Model: Strengths and Limitations

Strengths

- | Delivers useful software early
- | Provides visible progress
- | Supports prioritisation
- | Reduces initial delivery time
- | Allows feedback between releases
- | Distributes risk across several increments

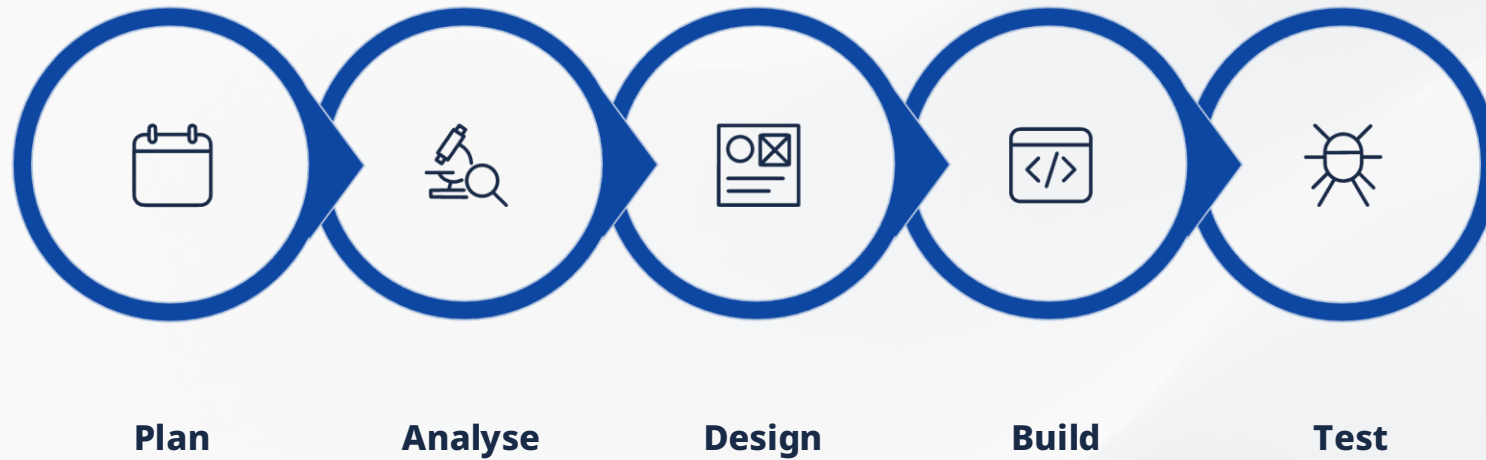
Limitations

- | Requires a strong overall architecture
- | Integration may become difficult
- | Dependencies among increments must be managed
- | Total cost may exceed that of a single planned release
- | Poor planning may produce inconsistent increments

ITERATIVE MODEL

Iterative Development Improves the Solution Repeatedly

In the Iterative model, an initial version of the complete or partial system is developed and then repeatedly refined.



Each iteration may improve functionality, usability, performance, architecture, reliability, and understanding of requirements.

Incremental and Iterative Are Different

Incremental Development	Iterative Development
Adds new functionality in parts	Improves existing functionality repeatedly
Product grows feature by feature	Product becomes more complete or refined
Each release adds another component	Each cycle revisits and improves the solution
Focuses on staged delivery	Focuses on learning and improvement

Many contemporary projects use both approaches together.

Iterative Model: Strengths and Limitations



Strengths

- | Supports learning and experimentation
- | Accommodates evolving requirements
- | Provides regular feedback
- | Identifies problems earlier
- | Supports progressive improvement
- | Reduces the risk of incorrect assumptions



Limitations

- | Final scope may remain uncertain
- | Repeated changes may increase cost
- | Project progress can be difficult to measure
- | Continuous stakeholder involvement is required
- | Uncontrolled iterations can delay completion

SPIRAL MODEL

Spiral Development Is Driven by Risk

The Spiral model combines iterative development with systematic risk management. The system develops through increasingly detailed cycles.



1. Define Objectives

Define objectives and alternatives for this cycle



2. Identify Risks

Identify and analyse risks for this cycle



3. Develop Solution

Develop and evaluate a solution or prototype



4. Plan Next Cycle

Plan the scope and activities for the next cycle

A Spiral Cycle in Practice

1

Cycle 1: Concept

Examine business feasibility, identify major project risks, produce a concept or feasibility prototype.

2

Cycle 2: Requirements

Clarify system requirements, analyse technical and security risks, develop a requirements prototype.

3

Cycle 3: Development

Build and test system components. Resolve integration and performance risks.

4

Cycle 4: Release

Validate, deploy and review the system.

Spiral Model: Strengths and Limitations

✓ Strengths

- | Places risk management at the centre
- | Supports changing requirements
- | Encourages prototyping
- | Suitable for large and complex systems
- | Allows stakeholder evaluation during each cycle
- | Supports progressive commitment of resources

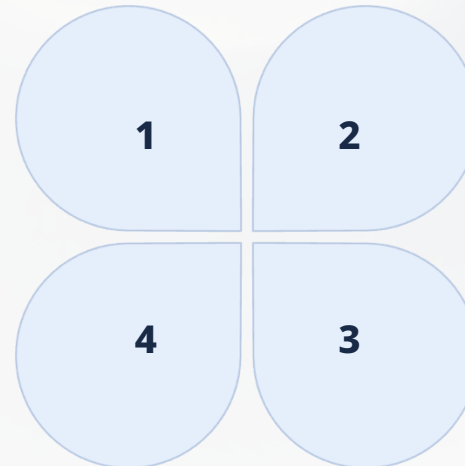
⚠ Limitations

- | More complex to manage
- | Requires risk-management expertise
- | Can be expensive
- | Project completion may be difficult to predict
- | Unsuitable for small and low-risk projects

Agile Responds to Change Through Short Cycles

Agile development uses short, repeated development cycles to deliver working software and obtain feedback.

People and Collaboration
Individuals and interactions over processes and tools



Working Software
Functional software over comprehensive documentation

Responsiveness
Responding to change over following a fixed plan

Customer Involvement
Customer collaboration and frequent delivery

Key point: Agile does not mean development without planning or documentation.

Core Agile Principles

01

Deliver valuable software early and frequently

02

Welcome necessary requirement changes

03

Work closely with customers and users

04

Organise work into short iterations

05

Maintain regular communication

06

Build projects around capable and motivated people

07

Continuously improve technical quality

08

Reflect and improve the development process

Common Agile Approaches

Approach	Main Emphasis
Scrum	Iterative delivery and team coordination
Extreme Programming	Engineering practices and code quality
Kanban	Visual workflow and work-in-progress control
Lean Software Development	Elimination of waste and value delivery
Feature-Driven Development	Development organised around features
Crystal	People, communication and project context

Scrum is a management framework and may be combined with technical practices from other approaches.

Scrum Roles and Responsibilities



Product Owner

- Maximises product value
- Manages and prioritises the Product Backlog
- Clarifies product requirements
- Represents stakeholder interests



Scrum Master

- Supports correct use of Scrum
- Removes organisational obstacles
- Facilitates team improvement
- Coaches the team and organisation



Developers

- Plan and perform development work
- Create a usable product increment
- Maintain quality
- Adapt their daily plan to achieve the Sprint Goal

Scrum Events

Event	Main Purpose
Sprint	Produces a usable product increment
Sprint Planning	Establishes the Sprint Goal and selects work
Daily Scrum	Reviews progress and adapts the daily plan
Sprint Review	Inspects the increment with stakeholders
Sprint Retrospective	Improves the team's process and practices

A Sprint is a fixed-length period, usually one month or less.

SCRUM FRAMEWORK

Scrum Artefacts and Commitments

Scrum Artefact	Description	Commitment
Product Backlog	Ordered list of product requirements and improvements	Product Goal
Sprint Backlog	Selected work and plan for the current Sprint	Sprint Goal
Increment	Completed and integrated product output	Definition of Done

Definition of Done

A shared quality standard used to determine whether work is genuinely complete.

Key point: An unfinished or untested feature is not part of a completed increment.

MODEL SELECTION

Selecting the Appropriate Life-Cycle Model

Key Selection Factors

- Stability of requirements
- Project size and complexity
- Level of technical risk
- System criticality
- Customer availability
- Need for early delivery
- Regulatory and documentation requirements
- Team experience
- Contract type
- Technology maturity

Quick Reference Guide

Project Situation	Appropriate Approach
Stable requirements and formal approvals	Waterfall or V-Model
Unclear user requirements	Prototyping
Need for early functional releases	Incremental
Requirements require repeated refinement	Iterative
Large, complex and high-risk project	Spiral
Frequent change and active customer involvement	Agile / Scrum
Mixed stability and governance requirements	Hybrid approach

Final Case Question

A university wants a mobile student-services application. Requirements differ among faculties, student feedback is essential, integration risk is high, and senior management needs an initial release within four months.

Task: Recommend a life-cycle approach and justify your answer using at least four selection factors.